Java Access Modifiers

A complete beginner-to-intermediate guide with real-life examples — Syeda Ajbina Nusrat, Lecturer, CSE, UITS

1. What Are Access Modifiers?

Access modifiers are **keywords that control who can see and use** a class, variable, method, or constructor. Think of them as **permission levels** — just like how different people in a company have different levels of access to information.

■ **Real-world analogy:** Imagine a hospital.
• A **patient's medical record** is PRIVATE — only the doctor handling that patient can see it.
• The **hospital's cafeteria menu** is DEFAULT — visible to everyone inside the hospital building.
• The **emergency procedures manual** is PROTECTED — shared with staff and partner clinics.
• The **hospital's phone number** is PUBLIC — anyone in the world can access it.

There are **4 access modifiers** in Java:

Modifier	Keyword	Restriction Level	Accessible From
Private	private	Most Restrictive ■	Only within the same class
Default	(none)	Package Level ■	Only within the same package
Protected	protected	Inheritance Level ■	Same package + subclasses anywhere
Public	public	Least Restrictive ■	Everywhere — any class, any package

■ **Golden Rule (from Oracle docs):** Always use the **most restrictive** access level that still makes sense. Start with private, and only open up access when needed.

2. private — The Vault ■

■ **private** = accessible ONLY within the class where it is declared. Not even subclasses or classes in the same package can access it.

Key facts:

- Most restrictive — maximum protection
- Used to **hide sensitive data** (encapsulation)
- Private fields are accessed from outside only through **getter/setter** methods
- Classes and interfaces themselves CANNOT be private (only their members)

Real-life scenario: A BankAccount class — your PIN and balance should never be directly touchable from outside.

```
class BankAccount {
  private String accountNumber;
  private double balance;
  private String pin;
```

```
  public BankAccount(String accNum, double initialBalance, String pin) {
    this.accountNumber = accNum;
    this.balance = initialBalance;
    this.pin = pin;
  }

  // Getter — controlled READ access
  public double getBalance() { return balance; }

  // No direct setter for balance — only through safe methods
  public void deposit(double amount) {
    if (amount > 0) balance += amount;
  }

  // Private helper — only used internally
  private boolean validatePin(String inputPin) {
    return this.pin.equals(inputPin);
  }

  public boolean withdraw(double amount, String inputPin) {
    if (validatePin(inputPin) && amount <= balance) {
      balance -= amount; return true;
    }
    return false;
  }
}

BankAccount acc = new BankAccount("ACC001", 5000, "1234");
// acc.balance = 99999; ■ COMPILE ERROR — private!
// acc.pin = "0000"; ■ COMPILE ERROR — private!
acc.deposit(500); // ■ OK — public method
System.out.println(acc.getBalance()); // ■ Output: 5500.0
```

■ **Why not just make everything public?** If balance were public, any other class could write **acc.balance = 1000000** and bypass all your security checks completely!

What happens if you try to access private from another class:

```
class Hacker {
  public static void main(String[] args) {
    BankAccount acc = new BankAccount("ACC001", 5000, "1234");
    System.out.println(acc.balance); // ■ ERROR: balance has private access
    System.out.println(acc.pin); // ■ ERROR: pin has private access
  }
}
```

3. default (package-private) — The Office Floor ■

■ **default** = no keyword written. Accessible only within the **same package**. Think of a package as a folder/department — everyone in that folder can see it, outsiders cannot.

Key facts:

- Applied automatically when you write NO modifier keyword
- Visible to all classes within the same package only
- Also called **package-private**
- Useful for utility/helper classes meant only for internal package use

Real-life scenario: A shopping app — internal helper classes used only within the 'payment' package.

```
// Package: com.shop.payment

// This class has DEFAULT access — only usable inside com.shop.payment
class PaymentValidator {
  boolean isValidCard(String cardNumber) {
    return cardNumber.length() == 16;
  }
  boolean isValidExpiry(String expiry) {
    return expiry.matches("\\d{2}/\\d{2}");
  }
}

// Also in com.shop.payment — CAN access PaymentValidator ■
public class CreditCardPayment {
  public void processPayment(String card, String expiry) {
    PaymentValidator v = new PaymentValidator();
    if (v.isValidCard(card) && v.isValidExpiry(expiry)) {
      System.out.println("Payment approved!");
    }
  }
}

// In com.shop.orders — DIFFERENT package — CANNOT access ■
// PaymentValidator v = new PaymentValidator(); // ERROR!
```

■ **When to use default:** When you have helper classes or utility methods that are implementation details — they support the public API but should not be used directly by outsiders.

4. protected — The Family Share ■

■■■■■ **protected** = accessible within the same package AND by any subclass (child class) even if it's in a different package. Think of it as sharing something with your family — not strangers.

Key facts:

- More open than default — subclasses in OTHER packages can also access
- Cannot be applied to top-level classes or interfaces
- A subclass can **override** a protected method but cannot make it more restrictive (e.g. private)
- Key tool for **inheritance-based designs**

Real-life scenario: A game engine — a base 'Character' class shares protected combat logic with all character subclasses.

```
// Package: com.game.engine
public class Character {
  protected String name;
  protected int health;
  protected int attackPower;

  public Character(String name, int health, int attackPower) {
    this.name = name;
    this.health = health;
    this.attackPower = attackPower;
  }

  // Protected — subclasses can use & override this
  protected void attack(Character target) {
    target.health -= this.attackPower;
    System.out.println(name + " attacks " + target.name);
  }

  // Protected — shared with subclasses only
  protected void heal(int amount) {
    health += amount;
    System.out.println(name + " healed " + amount + " HP");
  }
}

// Package: com.game.characters — DIFFERENT package, but SUBCLASS
public class Warrior extends Character {
  private int armor;

  public Warrior(String name, int armor) {
    super(name, 150, 30); // ■ accessing protected fields via super
    this.armor = armor;
  }

  // Overrides protected method — adds shield bonus
  @Override
  protected void attack(Character target) {
    super.attack(target); // base attack
    System.out.println(name + " shield-bashes for extra 10!");
    target.health -= 10;
  }
}

// Usage:
Warrior w = new Warrior("Thor", 50);
Character enemy = new Character("Goblin", 80, 10);
w.attack(enemy); // ■ Warrior can call protected attack()
```

■ **protected vs default:** Default = same package only. Protected = same package + ALL subclasses even in other packages. Protected is specifically designed for inheritance.

5. public — The Open Door ■

■ **public** = accessible from ANYWHERE — any class, any package, anywhere in the program. No restrictions at all. Use it for things that are meant to be the 'face' of your class.

Key facts:

- Least restrictive — widest scope
- Used for APIs, constructors, and methods you want the world to use
- Public fields are **discouraged** — prefer private fields + public getters/setters
- main() must always be public — Java needs to call it from outside

Real-life scenario: A weather service API — methods that any developer can call.

```
// Package: com.weather.api
public class WeatherService {
  private double temperature; // hidden internal data
  private String condition; // hidden internal data
  private double humidity;

  public WeatherService(String city) {
    // fetch weather data internally...
    this.temperature = 28.5;
    this.condition = "Sunny";
    this.humidity = 65.0;
  }

  // PUBLIC API — anyone can call these
  public double getTemperature() { return temperature; }
  public String getCondition() { return condition; }
  public double getHumidity() { return humidity; }

  public String getSummary() {
    return condition + ", " + temperature + "°C, Humidity: " + humidity + "%";
  }
}

// From ANY package, ANY class:
WeatherService ws = new WeatherService("Dhaka");
System.out.println(ws.getSummary()); // ■ Output: Sunny, 28.5°C, Humidity: 65.0%
```

■ **Don't over-use public!** Making everything public breaks encapsulation. If an internal method changes, all external code using it breaks too. Only make things public if they are genuinely part of your intended API.

6. The Big Comparison Table

This is the most important table to remember. ■ = accessible, ■ = not accessible.

Where accessed from	private	default (no keyword)	protected	public
Same class	■	■	■	■
Same package (different class)	■	■	■	■
Different package (subclass)	■	■	■	■
Different package (non-subclass)	■	■	■	■

Aspect	private ■	default ■	protected ■	public ■
Keyword	private	(nothing)	protected	public
Restriction	Most restrictive	Package only	Package + subclasses	No restriction
Used for	Sensitive data, internal helpers	Package-internal tools	Inheritance sharing	APIs, public interfaces
Getter/Setter?	Always needed	Sometimes	Rarely	No
Can class use it?	No	Yes	No (top-level)	Yes

7. Getters & Setters — The Private Field's Best Friend

When a field is private, the standard way to allow **controlled access** from outside is through **getter** (read) and **setter** (write) methods. This lets you add **validation logic** before allowing changes.

```
class Student {
  private String name;
  private int age;
  private double gpa;

  // GETTER — read only, no risk
  public String getName() { return name; }
  public int getAge() { return age; }
  public double getGpa() { return gpa; }

  // SETTER with validation — controlled write
  public void setName(String name) {
    if (name != null && !name.isEmpty())
      this.name = name;
    else System.out.println("Invalid name!");
  }
  public void setAge(int age) {
    if (age >= 1 && age <= 150) this.age = age;
    else System.out.println("Invalid age!");
  }
  public void setGpa(double gpa) {
    if (gpa >= 0.0 && gpa <= 4.0) this.gpa = gpa;
```

```
    else System.out.println("GPA must be 0.0 - 4.0!");
  }
}

Student s = new Student();
s.setName("Alice"); // ■ valid
s.setAge(300); // ■ prints 'Invalid age!' — setter blocked it
s.setGpa(3.8); // ■ valid
System.out.println(s.getName() + " GPA: " + s.getGpa()); // Alice GPA: 3.8
```

■ **Why setters are better than public fields:** If age were public, someone could write **s.age = -999** with no checks at all. The setter guarantees valid data always.

8. Full Real-Life Example — Online Store System

This example brings all 4 modifiers together in a realistic online store scenario. Notice how each modifier plays its role:

```
// ■■ Package: com.store.core ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public class Product { // public — anyone can use
  public String name; // public — freely readable
  protected double basePrice; // protected — subclasses can use
  double stockCount; // default — only store package sees this
  private String supplierCode; // private — internal only

  public Product(String name, double price, double stock, String code) {
    this.name = name;
    this.basePrice = price;
    this.stockCount = stock;
    this.supplierCode = code;
  }

  public double getPrice() { return basePrice; } // public getter
  private String getSupplierCode() { return supplierCode; } // internal only
}

// ■■ Package: com.store.products (different package, subclass) ■■
public class DiscountedProduct extends Product {
  private double discountPercent;

  public DiscountedProduct(String name, double price, double stock,
                           String code, double discount) {
    super(name, price, stock, code);
    this.discountPercent = discount;
  }

  @Override
  public double getPrice() {
    // ■ can access basePrice — it's protected
    return basePrice * (1 - discountPercent / 100);
  }
}
```

```
// ■■ From anywhere ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
DiscountedProduct p = new DiscountedProduct("Laptop",1000,50,"SUP01",20);
System.out.println(p.name); // ■ public
System.out.println(p.getPrice()); // ■ public method → Output: 800.0
// System.out.println(p.supplierCode); ■ private
// System.out.println(p.stockCount); ■ default — different package
```

9. When to Use Which Modifier

Situation	Best Choice	Why
Instance variables (fields)	private	Always hide data; expose via getters/setters
Internal helper methods	private	No need to expose implementation details
Main API methods for public use	public	Designed to be called by anyone
Class constructor	public	Others need to create objects
Methods shared with child classes	protected	Inheritance — subclasses need it, outsiders don't
Utility classes inside a package	default	Package tools, not part of public API
Constants (final static values)	public	Math.PI, Integer.MAX_VALUE style
Sensitive data (password, PIN, key)	private	Must never be directly accessible
Framework base class methods	protected	Let subclasses customize behavior

10. Common Mistakes to Avoid

■ Mistake: Making all fields public

```
public int age; // ■ Anyone can set age = -500 directly
private int age; // ■ Force validation through setAge()
```

■ Mistake: Confusing default with public

```
// default class in file MyHelper.java:
class MyHelper { } // ■ only same-package access — other packages can't import it
public class MyHelper { } // ■ if you need it everywhere
```

■ Mistake: Making protected when private would do

```
protected String password; // ■ subclasses can read the password!
private String password; // ■ even child classes can't see it
```

■ Mistake: Forgetting that private is NOT inherited

```
class Animal { private int dna; }
class Dog extends Animal {
  void show() { System.out.println(dna); } // ■ ERROR — private not inherited
}
// Fix: use protected int dna or a getter
```

Quick Reference Card

Question	Answer
Most restrictive modifier?	private — same class only
Which modifier is default when nothing written?	default (package-private)
Can private be inherited?	No — never
Can protected be used by subclass in another package?	Yes — that's its main purpose
Should fields usually be public?	No — use private + getters/setters
Can a top-level class be private?	No — only public or default
Can a top-level class be protected?	No — only public or default
Which modifier should main() use?	Always public
Order from most to least restrictive?	private → default → protected → public

---

Summary compiled from lecture slides + Oracle Java Docs + Baeldung + GeeksForGeeks — Syeda Ajbina Nusrat, Lecturer, CSE, UITS